
Security Review Report
NM-1094 - Nuxaris-CBNB Contracts



NETHERMIND
SECURITY

(October 08, 2026)

Contents

1 Executive Summary	2
2 Audited Files	3
3 Summary of Issues	3
4 System Overview	4
4.1 Components	4
4.2 Deposit flow (BNB to cBNB)	4
4.3 Redemption flow (cBNB to BNB)	4
5 Risk Rating Methodology	5
6 Issues	6
6.1 [Low] Deposits have no depositor-side bound on fees, so a concurrent fee update can silently consume part of the deposit	6
6.2 [Info] Deposit message IDs are only unique within a single vault deployment	7
7 Documentation Evaluation	8
8 Test Suite Evaluation	9
8.1 Tests Output	9
9 About Nethermind	11

1 Executive Summary

This document presents the results of the security review conducted by [Nethermind Security](#) for **Nuxaris LLC**. The focus of the engagement has been the CBNBRouter and CBNBVault contracts deployed on BNB Smart Chain, which hold the BNB backing cBNB, a 1:1 representation of BNB issued on the Canton Network. On BSC mainnet, CBNBRouter is deployed at [0x74B4052AE403E0787DFC1dA8a1247dD950755C28](#) and CBNBVault at [0x38c147415e59196aEBf3010dd0BAA9A7e5Bf9196](#).

Whitelisted depositors lock BNB on BSC, an off-chain relayer mints the same amount of cBNB on Canton after the deposit is finalized, and on redemption the relayer burns the cBNB on Canton before a multisig releases the BNB on BSC.

The audit comprises 398 lines of Solidity code. **The audit was performed using** (a) manual analysis of the codebase, and (b) automated analysis tools.

Along this document, we report two points of attention, where one is classified as Low , and one is classified as Informational severity. The issues are summarized in Fig. 1.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 presents the summary of issues. Section 4 presents the System Overview. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 presents the test suite evaluation and automated tools used. Section 9 concludes the document.

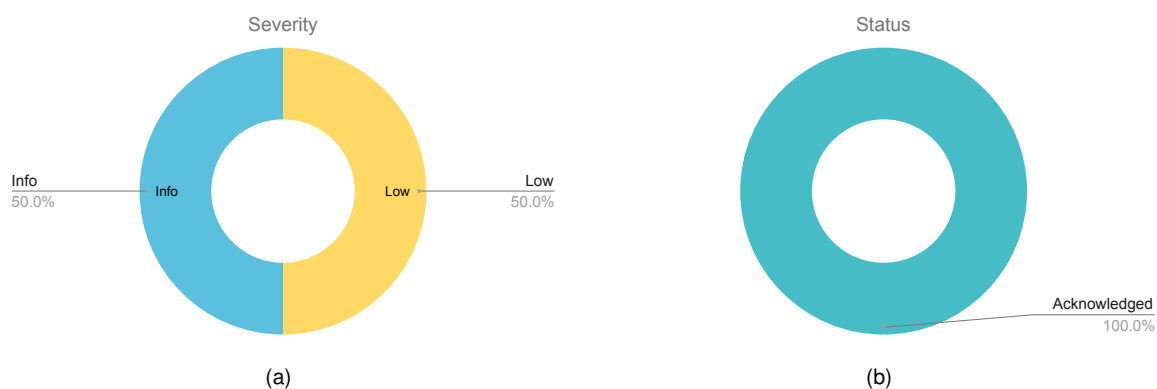


Fig. 1: Distribution of issues: Critical (0), High (0), Medium (0), Low (1), Undetermined (0), Informational (1), Best Practices (0).
Distribution of status: Fixed (0), Acknowledged (2), Mitigated (0), Unresolved (0)

Summary of the Audit

Audit Type	Security Review
Initial Report	October 08, 2026
Final Report	October 08, 2026
Initial Commit	ed6b882
Final Commit	ed6b882
Documentation Assessment	High
Test Suite Assessment	High

2 Audited Files

	Contract	LoC	Comments	Ratio	Blank	Total
1	src/CBNBRouter.sol	252	84	33%	65	400
2	src/CBNBVault.sol	146	49	33%	35	230
	Total	398	133	33.4%	100	630

3 Summary of Issues

	Finding	Severity	Update
1	Deposits have no depositor-side bound on fees, so a concurrent fee update can silently consume part of the deposit	Low	Acknowledged
2	Deposit message IDs are only unique within a single vault deployment	Info	Acknowledged

4 System Overview

The system consists of two contracts on BNB Smart Chain, **CBNBRouter** and **CBNBVault**, operated by three Safe multisigs: the Admin Safe, which holds the admin role on both contracts, the Manager Safe, which holds the manager role on the router, and the Relayer Safe, which holds the relayer role on the router. On BSC mainnet, CBNBRouter is deployed at `0x74B4052AE403E0787DFC1dA8a1247dD950755C28` and CBNBVault at `0x38c147415e59196aEBf3010dd0BAA9A7e5Bf9196`.

4.1 Components

CBNBRouter is the user-facing entry point. It restricts deposits to a whitelist that the Manager Safe maintains, deducts a fixed fee and a proportional fee capped at 10%, enforces a minimum net amount, and forwards the remainder to the vault. The fees stay in the router as a tracked balance that only the Admin Safe can collect, and the native-BNB sweep refuses any amount that would dip into it. The router also forwards release requests from the Relayer Safe to the vault, holds the destination chain and token configuration, and exposes separate pause flags for deposits and withdrawals that the Manager can set but only the Admin can clear.

CBNBVault is the custody contract and accepts calls only from the router the Admin Safe has configured. On each deposit it derives a `messageId` from the depositor, chain ID, an incrementing nonce, the net amount, the destination and the recipient, and emits it for the relayer. On each release it marks the `messageId` as processed so no request can be paid twice, enforces an optional per-UTC-day release limit, and transfers the BNB. Its sweep function moves ERC-20 tokens only and cannot move native BNB.

The router's reference to the vault is immutable. The vault's reference to the router is a storage variable that only the vault admin can change.

4.2 Deposit flow (BNB to cBNB)

- The Manager Safe whitelists the buyer's BSC address after off-chain identity and compliance checks.
- The buyer calls `CBNBRouter.deposit(recipient)` with BNB as `msg.value`. `recipient` is the buyer's Canton party ID. The router requires the caller to be whitelisted, deposits to be unpaused, and the destination to be configured.
- The router deducts the fixed `cryptoFee` and the proportional `protocolFeeBps` (capped at 10%), requires the remainder to be at least `minAmount`, adds the fees to `feeBalance`, and forwards the remainder to `CBNBVault.deposit`.
- The vault computes a `messageId` from the depositor, chain ID, an incrementing nonce, the net amount, the destination and the recipient, and emits `Deposit`.
- After BSC finality, the relayer verifies the event on several RPC providers, checks that total minted supply plus this deposit does not exceed the vault balance, and mints the same amount of cBNB on Canton.

4.3 Redemption flow (cBNB to BNB)

- The holder submits a redemption request off-chain and transfers the exact cBNB amount to the `cbnb-redeem` party with the request ID as memo.
- The relayer matches the transfer to the request, a person approves it, and the relayer burns the cBNB on Canton. The supply decreases before any BNB moves.
- The relayer prepares `finalizeWithdraw(messageId, destination, amount)` with `messageId = keccak256("cbnb-redeem:" + requestId)`. Two Relayer Safe owners sign and execute it.
- The router checks that withdrawals are unpaused and the caller holds `RELAYER_ROLE`, then calls `CBNBVault.release`.
- The vault marks the `messageId` as processed (reverting if already seen), enforces the per-UTC-day release limit if it is non-zero, transfers the BNB and emits `Withdraw`.

5 Risk Rating Methodology

The risk rating methodology used by [Nethermind Security](#) follows the principles established by the [OWASP Foundation](#). The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

- a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;
- b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;
- c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

- a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;
- b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;
- c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	Medium	High
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, [Nethermind Security](#) also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

- a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;
- b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;
- c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

6.1 [Low] Deposits have no depositor-side bound on fees, so a concurrent fee update can silently consume part of the deposit

File(s): `src/CBNBRouter.sol`

Description: Whitelisted depositors lock BNB by calling the `deposit(...)` function of the CBNBRouter contract. They send the BNB as `msg.value`, and the router deducts two fees whose values are read from storage at execution time: the fixed `cryptoFee` and the proportional `protocolFeeBps`. The router forwards the remainder to the vault. The relayer mints exactly this remainder as cBNB on Canton. The fees stay in the router as `feeBalance`, which only the admin can collect.

```

1  function deposit(bytes calldata _recipient)
2      // ...
3  {
4      // ...
5      if (msg.value <= cryptoFee) {
6          revert InvalidCryptoFeeAmount();
7      }
8
9      // @audit-issue The fee values are read at execution time, and the depositor cannot bound them.
10     uint256 depositAmount = msg.value - cryptoFee;
11
12     uint256 protocolFee = _calculateFee(depositAmount);
13
14     uint256 amountAfterFees = depositAmount - protocolFee;
15
16     // @audit The only check on the net amount is the protocol-controlled minimum.
17     if (amountAfterFees < minAmount) {
18         revert AmountBelowMinimum();
19     }
20
21     uint256 totalFees = cryptoFee + protocolFee;
22     if (totalFees > 0) {
23         feeBalance += totalFees;
24     }
25
26     messageId = vault.deposit{value: amountAfterFees}(/* ... */);
27 }

```

The Manager Safe or the Admin Safe can change both fees at any time through the `setFees(...)` function. The change takes effect immediately, with no delay. Only `protocolFeeBps` has an upper bound (10%); `cryptoFee` has none.

```

1  function setFees(uint256 _newCryptoFee, uint16 _newProtocolFeeBps)
2      external
3      onlyManagerOrAdmin
4  {
5      if (_newProtocolFeeBps > MAX_PROTOCOL_FEE_BPS) {
6          revert FeeTooHigh();
7      }
8      // @audit-issue There is no upper bound on the fixed fee and no delay before it applies.
9      cryptoFee = _newCryptoFee;
10     protocolFeeBps = _newProtocolFeeBps;
11     emit FeesUpdated(_newCryptoFee, _newProtocolFeeBps);
12 }

```

The `deposit(...)` function does not let the depositor state the fees they agreed to or the minimum amount they expect to have bridged. The only check on the net amount is `minAmount`, and the protocol sets it, not the depositor. If a fee update is executed before a pending deposit, whether in the same block or while the deposit is waiting to be mined, the deposit still succeeds. It is charged the new fees as long as the remainder stays at or above `minAmount`. This needs no malicious intent: any routine fee change, such as the first move away from the current `0 / 0` configuration, applies to every deposit that was signed under the old fees and is mined after the update.

Recommendation(s): Consider giving depositors a way to make sure that a deposit is only charged the fees they agreed to, for example by binding each deposit to an expected fee or a minimum bridged amount. If the deployed contracts are to stay as they are, consider an operational procedure that keeps fee updates from overlapping with pending deposits (for example, pausing deposits, waiting until pending deposits are mined or dropped, and only then updating the fees), and communicating fee changes to whitelisted depositors before they apply. Also consider placing an upper bound on `cryptoFee`.

Status: Acknowledged

Update from the client: Only Nuxaris-controlled addresses are whitelisted depositors, therefore this cannot affect external depositors.

6.2 [Info] Deposit message IDs are only unique within a single vault deployment

File(s): `src/CBNBVault.sol`

Description: Whitelisted depositors bridge BNB to Canton by calling the `deposit(...)` function of the `CBNBRouter` contract, which forwards the net amount to the `deposit(...)` function of the `CBNBVault` contract. The vault derives a `messageId` for every deposit and emits it in the `Deposit` event. The off-chain relayer watches these events and, once the block is finalized, mints the same amount of cBNB on Canton. The `NatSpec` describes `messageId` as the identifier "used for idempotent processing", and the `README` states that it "is unique per deposit, because it includes the vault nonce".

```

1  /// @return messageId Unique message identifier used for idempotent processing
2  function deposit(...)
3      external
4      payable
5      onlyRouter
6      returns (bytes32 messageId)
7  {
8      // ...
9      messageId = _nextMessageId(_sender, msg.value, _dstChainID, _dstOutputToken, _recipient);
10
11     // @audit The relayer reads the messageId from this event.
12     emit Deposit(messageId, /* ... */);
13 }

```

The `_nextMessageId(...)` function computes the `messageId` as the hash of the depositor, the chain ID, the vault's nonce, the net amount, the destination and the recipient. The nonce is incremented on every deposit, so two deposits into the same vault always receive different IDs, even when all their other parameters are identical.

```

1  function _nextMessageId(
2      address _sender,
3      uint256 _amount,
4      uint256 _dstChainID,
5      bytes32 _dstOutputToken,
6      bytes memory _recipient
7  ) internal returns (bytes32) {
8      // @audit-issue The nonce is vault storage and starts at 0 in every new vault.
9      uint256 currentNonce = nonce++;
10     // @audit-issue None of the hashed fields identifies the vault instance.
11     return keccak256(
12         abi.encodePacked(
13             _sender,
14             block.chainid,
15             currentNonce,
16             _amount,
17             _dstChainID,
18             _dstOutputToken,
19             _recipient
20         )
21     );
22 }

```

However, this uniqueness only holds within a single vault instance. The nonce is a storage variable of the vault and starts at 0 in every newly deployed vault, and none of the hashed fields identifies the vault itself. If the vault is redeployed on the same chain for any reason, its nonce does not continue from the previous vault's nonce. The *k*-th deposit into the new vault then receives exactly the same `messageId` as the *k*-th deposit into the old vault whenever both have the same depositor, net amount, destination and recipient. This breaks the uniqueness and idempotency guarantee of `messageId`, so any off-chain infrastructure or process that relies on it, such as the relayer, would be working on a wrong assumption.

Recommendation(s): Consider accounting for this limitation in any future vault deployment or migration, so that deposit message IDs remain unique across all vault instances.

Status: Acknowledged

Update from the client: Only Nuxaris-controlled addresses are whitelisted depositors, therefore this cannot affect external depositors.

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- Technical whitepaper: A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- User manual: A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- Code documentation: Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- API documentation: API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- Testing documentation: Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- Audit documentation: Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about Nuxaris-LLC-CBNB contracts documentation

The codebase contains detailed Readme that explains the expected flows in the system, along with trust assumptions and known code limitations. Additionally, the development team provided answers to questions raised by the Nethermind Security team during the sync call.

The code is also accompanied with comprehensive Natspec documentation describing the purpose, inputs, and outputs for each function. Functions with more complex logic also have inline comments to improve code readability.

8 Test Suite Evaluation

8.1 Tests Output

```
forge test
[] Compiling...
No files changed, compilation skipped

Ran 10 tests for test/KnownLimitations.t.sol:KnownLimitationsTest
[PASS] test_AdminSafeCanDrainTheVault() (gas: 149960)
[PASS] test_DepositorAndRelayerLoopIsBoundedByTheDailyLimit() (gas: 196415)
[PASS] test_EmptyRecipientIsAccepted() (gas: 74491)
[PASS] test_ManagerCanBlockDepositsWithCryptoFeeButNotTakeFees() (gas: 142189)
[PASS] test_ManagerCanBlockDepositsWithMinAmount() (gas: 46114)
[PASS] test_RelayerSafeReleasesWithoutOnChainProofOfBurn() (gas: 200055)
[PASS] test_ReleaseAboveTheDailyLimitNeedsSplitOrHigherLimit() (gas: 312700)
[PASS] test_TwiceTheDailyLimitAroundMidnightUtc() (gas: 225925)
[PASS] test_WhitelistEventsEvenWithoutChange() (gas: 35837)
[PASS] test_WhitelistedAddressCanDepositTowardsAnyRecipient() (gas: 78641)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 5.74ms (1.21ms CPU time)

Ran 18 tests for test/Security.t.sol:SecurityTest
[PASS] test_CollectFeeChecks() (gas: 239991)
[PASS] test_DailyLimitZeroDisablesTheCheck() (gas: 122544)
[PASS] test_DepositMustExceedTheFixedFee() (gas: 61193)
[PASS] test_DepositPauseAndAdminUnpause() (gas: 88394)
[PASS] test_DepositRefusedWithoutDestinationChain() (gas: 39324)
[PASS] test_DustBelowOneCantonUnitStaysInTheVault() (gas: 75490)
[PASS] test_FinalizeWithdrawInputChecks() (gas: 101915)
[PASS] test_ForcedBnbStaysInTheVault() (gas: 83028)
[PASS] test_OnlyManagerOrAdminCanConfigure() (gas: 54526)
[PASS] test_ReentrancyIsBlocked() (gas: 445696)
[PASS] test_RejectingRecipientKeepsTheMessageIdUsable() (gas: 275562)
[PASS] test_RouterAdminCanBeRotatedButNotRemoved() (gas: 89219)
[PASS] test_RouterConstructorChecks() (gas: 334823)
[PASS] test_RouterSweepChecksAndPaths() (gas: 383900)
[PASS] test_VaultAdminFunctions() (gas: 85091)
[PASS] test_VaultConstructorRejectsZeroAdmin() (gas: 58989)
[PASS] test_VaultEntryPointsAreRouterOnly() (gas: 44489)
[PASS] test_WhitelistManagement() (gas: 41934)
Suite result: ok. 18 passed; 0 failed; 0 skipped; finished in 5.73ms (2.09ms CPU time)

Ran 19 tests for test/CBNB.t.sol:CBNBTest
[PASS] testFuzz_ValueConserved(uint96,uint64,uint16) (runs: 256, : 148457, ~: 151140)
[PASS] test_BelowMinimumRefused() (gas: 36778)
[PASS] test_DailyLimitHoldsThenResetsNextUtcDay() (gas: 263984)
[PASS] test_DepositLocksBnbInVaultAndEmitsRecipientInClear() (gas: 81891)
[PASS] test_DirectBnbToVaultOrRouterRefused() (gas: 29012)
[PASS] test_FeeCappedAtTenPercent() (gas: 15440)
[PASS] test_FeesStayInRouterAndMemberGetsNet() (gas: 148350)
[PASS] test_LastAdminCannotBeRemoved() (gas: 27379)
[PASS] test_LaunchDaySwitchIsOneCall() (gas: 22297)
[PASS] test_NoDestinationRefused() (gas: 39530)
[PASS] test_NonWhitelistedDepositorRefused() (gas: 27677)
[PASS] test_OnlyAdminRewiresVaultRouter() (gas: 15206)
[PASS] test_OnlyRelayerCanRelease() (gas: 85864)
[PASS] test_PausesAreSeparateAndManagerCannotUnpause() (gas: 190912)
[PASS] test_RejectedReleaseDoesNotBurnItsId() (gas: 121239)
[PASS] test_RelayerReleasesOnceOnly() (gas: 149162)
[PASS] test_RouterSweepCannotTakeFees() (gas: 130782)
[PASS] test_TwoIdenticalDepositsGetDistinctIds() (gas: 97646)
[PASS] test_VaultBnbCannotBeSweptByAdmin() (gas: 90218)
Suite result: ok. 19 passed; 0 failed; 0 skipped; finished in 14.93ms (15.77ms CPU time)
```

Ran 2 tests **for** test/Security.t.sol:CBNBInvariantTest

[PASS] invariant_RouterHoldsOnlyTheUncollectedFees() (runs: 256, calls: 128000, reverts: 165)

Contract	Selector	Calls	Reverts	Discards
CBNBHandler	collect	31855	0	0
CBNBHandler	deposit	32040	165	0
CBNBHandler	release	31952	0	0
CBNBHandler	setFees	32153	0	0

[PASS] invariant_VaultBalanceMatchesDepositsMinusReleases() (runs: 256, calls: 128000, reverts: 124)

Contract	Selector	Calls	Reverts	Discards
CBNBHandler	collect	32238	0	0
CBNBHandler	deposit	32000	124	0
CBNBHandler	release	31862	0	0
CBNBHandler	setFees	31900	0	0

Suite result: ok. 2 passed; 0 failed; 0 skipped; finished **in** 5.24s (10.05s CPU **time**)

Ran 4 **test** suites **in** 5.25s (5.27s CPU **time**): 49 tests passed, 0 failed, 0 skipped (49 total tests)

9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to [Nethermind](#) in order that [Nethermind](#) could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. [Nethermind](#) has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, [Nethermind](#) disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. [Nethermind](#) does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and [Nethermind](#) will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.